

ALGORITMA DAN STRUKTUR DATA(ASD)

TUGAS PRAKTIKUM 10 TREE



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C

PROGRAM STUDI TEKNIK INFORMATIKA

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TUGAS PRAKTIKUM ASD – TREE

Implementasikan Tree untuk membentuk pohon dimana node baru dengan nilai yang lebih kecil dari node parent menjadi child kiri, sebaliknya menjadi child kanan. Tambahkan fungsi kunjungan PreOrder, InOrder, dan PostOrder. Tambahkan juga fungsi untuk menghitung depth (kedalaman) dan size (jumlah node) menggunakan rekursi. Laporan dalam bentuk source code, hasil running program, dan analisis. Buatlah kesimpulan hasil praktikum

Jawaban:

Kode:

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. struct tree {
5.     int info;
6.     struct tree *kiri;
7.     struct tree *kanan;
8. };
9.
10. typedef struct tree *pohon;
11.
12. pohon baru(int data);
13. void sisip(pohon ph, pohon sp);
14. void preorder(pohon ph);
15. void inorder(pohon ph);
16. void postorder(pohon ph);
17. int hitung_size(pohon ph);
18. int hitung_depth(pohon ph);
19.
20. int main() {
21.     pohon root = NULL;
22.     pohon node_baru;
23.     int n, data;
24.
25.     printf("pembentukan binary search tree\n");
26.     printf("masukkan jumlah data yang ingin diinput: ");
27.     scanf("%d", &n);
28.
29.     for (int i = 0; i < n; i++) {
30.         printf("masukkan data ke-%d: ", i + 1);
31.         scanf("%d", &data);
32.
33.         if (root == NULL) {
34.             root = baru(data);
```

```

35.         } else {
36.             node_baru = baru(data);
37.             sisip(root, node_baru);
38.         }
39.     }
40.
41.     printf("\nhasil penelusuran:\n");
42.     printf("preorder  : "); preorder(root); printf("\n");
43.     printf("inorder   : "); inorder(root); printf("\n");
44.     printf("postorder  : "); postorder(root); printf("\n");
45.
46.     printf("\nanalisis pohon:\n");
47.     printf("jumlah simpul (size) : %d\n", hitung_size(root));
48.     printf("kedalaman (depth)    : %d\n", hitung_depth(root));
49.
50.     return 0;
51. }
52.
53. pohon baru(int data) {
54.     pohon br = (pohon)malloc(sizeof(struct tree));
55.     if (br != NULL) {
56.         br->info = data;
57.         br->kiri = NULL;
58.         br->kanan = NULL;
59.     }
60.     return br;
61. }
62.
63. void sisip(pohon ph, pohon sp) {
64.     pohon p = ph;
65.     pohon q = ph;
66.
67.     while (q != NULL) {
68.         p = q;
69.         if (sp->info == p->info) {
70.             printf("data %d sudah ada di dalam tree!\n", sp->info);
71.             return;
72.         } else if (sp->info < p->info) {
73.             q = p->kiri;
74.         } else {
75.             q = p->kanan;
76.         }
77.     }

```

```

78.
79.     if (sp->info < p->info) {
80.         p->kiri = sp;
81.     } else {
82.         p->kanan = sp;
83.     }
84. }
85.
86. void preorder(pohon ph) {
87.     if (ph != NULL) {
88.         printf("%d ", ph->info);
89.         preorder(ph->kiri);
90.         preorder(ph->kanan);
91.     }
92. }
93.
94. void inorder(pohon ph) {
95.     if (ph != NULL) {
96.         inorder(ph->kiri);
97.         printf("%d ", ph->info);
98.         inorder(ph->kanan);
99.     }
100. }
101.
102. void postorder(pohon ph) {
103.     if (ph != NULL) {
104.         postorder(ph->kiri);
105.         postorder(ph->kanan);
106.         printf("%d ", ph->info);
107.     }
108. }
109.
110. int hitung_size(pohon ph) {
111.     if (ph == NULL) return 0;
112.     return 1 + hitung_size(ph->kiri) + hitung_size(ph->kanan);
113. }
114.
115. int hitung_depth(pohon ph) {
116.     if (ph == NULL) return -1;
117.     int depth_kiri = hitung_depth(ph->kiri);
118.     int depth_kanan = hitung_depth(ph->kanan);
119.

```

```

120.         if (depth_kiri > depth_kanan) {
121.             return depth_kiri + 1;
122.         } else {
123.             return depth_kanan + 1;
124.         }
125.     }

```

Hasil Kode:

```

pembentukan binary search tree
masukkan jumlah data yang ingin diinput: 5
masukkan data ke-1: 23
masukkan data ke-2: 13
masukkan data ke-3: 67
masukkan data ke-4: 47
masukkan data ke-5: 39

hasil penelusuran:
preorder  : 23 13 67 47 39
inorder   : 13 23 39 47 67
postorder : 13 39 47 67 23

analisis pohon:
jumlah simpul (size) : 5
kedalaman (depth)   : 3

```

Analisis Program:

Program ini diatur dari dalam blok main yang meminta input data, lalu alur utamanya bergantung pada fungsi baru dan sisip. Pertama, fungsi baru akan mulai bekerja dengan menyiapkan ruang memori, mengisi angkanya, dan memastikan cabang kiri dan kanannya dalam keadaan kosong. Setelah memori siap, fungsi sisip akan mengambil alih dengan mengecek posisi dari bagian akar terlebih dahulu; ia mengecek apakah angka baru lebih kecil dari simpul saat ini, jika iya program turun ke cabang kiri, dan jika lebih besar turun ke cabang kanan. Pengecekan ini diulang terus-menerus sampai program menemukan ujung cabang yang kosong untuk menempelkan data baru tersebut.

Untuk tahap pembacaan dan perhitungan, program mengandalkan fungsi rekursi (fungsi yang memanggil dirinya sendiri berulang kali). Fungsi inorder mulai dengan menelusuri cabang paling kiri sampai mentok, mencetak angka, lalu bergeser ke kanan, sehingga hasil cetakannya terurut dari kecil ke besar (sedangkan preorder dan postorder hanya berbeda urutan bacanya saja). Terakhir, fungsi hitung_size bekerja dengan menjumlahkan total simpul dari cabang kiri dan kanan ditambah satu, sementara fungsi hitung_depth mulai dengan mengukur kedalaman masing-masing cabang, mengecek mana yang lebih panjang, lalu mengambil nilai terdalam tersebut sebagai hasil akhirnya.

Kesimpulan:

Praktikum ini membuktikan bahwa struktur pohon dengan memori dinamis (*linked list*) sangat efektif untuk mengelola data yang bertingkat tanpa batasan kapasitas awal yang kaku. Aturan penyusunan pohon biner pencarian yang menempatkan nilai lebih kecil ke kiri dan nilai lebih besar ke kanan membuat proses penyortiran data menjadi sangat efisien, yang mana bisa langsung dibuktikan dari hasil keluaran metode *inorder* yang otomatis mencetak data acak menjadi berurutan secara rapi.

Selain itu, penggunaan konsep rekursi terbukti sangat krusial dalam mempermudah manipulasi struktur data yang bercabang banyak. Alih-alih menggunakan perulangan biasa yang bisa menjadi sangat rumit, rekursi membuat proses penelusuran sela-sela cabang, perhitungan total data, hingga pengukuran kedalaman pohon memiliki alur kerja yang jauh lebih ringkas, sederhana, dan mudah dipahami secara logika.